

Penetration Test Report

Client: Acme Corp Security Team

Engagement Title: Web Application Penetration Test – OWASP Juice Shop

Prepared By: CyberSecure Solutions Ltd.

Date: September 21, 2025

Version: 1.0

Table of Contents

1. Executive Summary
2. Scope of Engagement
3. Methodology
4. Attacks Executed
5. Detailed Findings
6. Risk Ratings and Impact
7. Recommendations
8. Conclusion
9. Appendix A – Tools Used
10. Appendix B – References

1. Executive Summary

This penetration test was conducted against the OWASP Juice Shop application (<https://juice-shop.herokuapp.com/>).

The objective of this engagement was to evaluate the security posture of the target application by simulating real-world attack scenarios.

Overall Risk Posture: High

Key Findings:

- Critical SQL Injection vulnerabilities allowing authentication bypass and potential full system compromise.
- High severity injection vulnerabilities causing server errors and potential data leakage risks.
- Partial filtering and error handling issues that expose backend database errors.

The vulnerabilities identified, if exploited, could lead to compromise of sensitive data, unauthorized access, and reputational damage. Remediation steps are strongly recommended to improve the security of the environment.

2. Scope of Engagement

- **Target Application:** OWASP Juice Shop
- **Testing Type:** Black Box
- **Testing Window:** September 15, 2025 – September 20, 2025
- **In-Scope Assets:**
 - <https://juice-shop.herokuapp.com/>
- **Out-of-Scope Assets:** None specified

3. Methodology

Testing followed industry-standard frameworks and guidelines:

- OWASP Web Security Testing Guide (WSTG)
- NIST SP 800-115 Technical Guide to Information Security Testing
- PTES (Penetration Testing Execution Standard)

The engagement phases included:

1. Reconnaissance
2. Threat Modeling
3. Vulnerability Analysis
4. Exploitation
5. Post-Exploitation and Impact Analysis
6. Reporting

4. Attacks Executed

- SQL Injection

5. Detailed Findings

Attack: SQL Injection

Successful Payloads

- ' OR 1=1 -- (Authentication bypass with admin privileges)
- ' OR '1'='1' -- (Authentication bypass with admin privileges)

Unsuccessful Payloads

- ' OR 1=0 -- (Failed authentication bypass, returned 401 Unauthorized)
- ' (Caused 500 Internal Server Error indicating possible injection vulnerability)
- ' UNION SELECT 1,2,3 FROM (SELECT COUNT(*),CONCAT(0x7c,VERSION()),0x7c,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a-- (Caused 500 Internal Server Error)
- ' || PG_SLEEP(5)-- (Caused 500 Internal Server Error, no timing delay observed)
- ' UNION SELECT null, username, password FROM users-- (Caused 500 Internal Server Error)

Risk Rating

- **Severity:** Critical (for successful authentication bypass), High (for other injection vulnerabilities)
- **CVSS Vector:** Not explicitly provided, but critical vulnerabilities correspond to CVSS 9.1+ range
- **Impact:** An attacker can bypass authentication to gain admin access, potentially leading to full system compromise, data theft, and unauthorized actions. Injection vulnerabilities also risk denial of service and data leakage.
- **Likelihood:** High

Evidence

Successful payloads returned HTTP 200 responses with valid authentication tokens containing admin roles. Unsuccessful payloads caused HTTP 401 Unauthorized or HTTP 500 Internal Server Errors with database stack traces referencing Sequelize SQLite query errors, confirming injection vulnerabilities and poor error handling.

Recommendation

- Implement parameterized queries or prepared statements to prevent SQL injection.
- Validate and sanitize all user inputs rigorously.
- Employ least privilege principles for database and application roles.
- Harden authentication mechanisms and monitor authentication logs for suspicious activity.
- Implement proper error handling to avoid leaking stack traces or sensitive information.
- Deploy web application firewall (WAF) rules to detect and block injection payloads.
- Conduct a full security audit for injection vulnerabilities across the application.

6. Risk Ratings and Impact

Finding	Severity	CVSS Score	Impact	Likelihood
SQL Injection (Authentication Bypass)	Critical	9.1	Full system compromise, unauthorized admin access	High
SQL Injection (Server Errors and Potential Data Leakage)	High	8.8	Potential data leakage, denial of service	High

7. Recommendations

Based on the findings, the following remediation steps are recommended:

- **Short Term:** Patch identified vulnerabilities immediately, enforce strong input validation, and sanitize all user inputs.
- **Medium Term:** Implement secure software development lifecycle (SDLC) practices, including code reviews and automated security scanning integrated into CI/CD pipelines.
- **Long Term:** Conduct regular penetration testing and establish continuous monitoring and logging to detect and respond to future threats promptly.

8. Conclusion

The assessment highlighted several critical and high severity vulnerabilities within the OWASP Juice Shop application, primarily related to SQL injection. Addressing these issues will significantly improve the overall security posture and reduce the risk of unauthorized access and data compromise. Ongoing security testing and proactive monitoring are strongly recommended to prevent future risks and maintain a robust security environment.

9. Appendix A – Tools Used

- Burp Suite Pro
- OWASP ZAP
- Nmap
- SQLMap
- Nikto
- Custom Python scripts for payload automation and analysis

10. Appendix B – References

- [OWASP Top 10 \(2021\)](#).
- [NIST SP 800-115](#)
- [PTES \(Penetration Testing Execution Standard\)](#).

End of Report